PNLIO 2031

Despliegue unificado ADÁN 2031 + EVA 2031

Código fuente Python en un solo archivo

Autor: Gonzalo Mauricio De la Rivera Arellano

Licencia: Apache-2.0

Este PDF contiene el código adjunto como referencia documental. La ejecución no se realiza durante la generación del documento.

1 Resumen

El archivo implementa un instalador unificado para un ecosistema local PNLIO que incluye ADÁN 2031, SOLEDAD Kernel v1.3 y EVA 2031 v1.7.2. Automatiza la preparación de dependencias, modelos locales, entorno virtual, servicios y archivos de ejecución.

1.1 Alcance declarado

- ADÁN 2031: chat local con Ollama para consultar documentos PDF, Word, Excel, texto y CSV.
- SOLEDAD Kernel v1.3: memoria SQLite FTS5, voz, visión y un núcleo robótico supervisado.
- EVA 2031: dashboard Gradio con RAG, visión, voz, oído y una capa CSP.

1.2 Nota de seguridad y operación

El propio código declara que los núcleos robóticos operan en simulación y que el hardware real permanece bloqueado hasta contar con un driver verificado. Este documento no constituye certificación de seguridad, validación de hardware ni autorización para operar un robot físico.

1.3 Requisitos indicados por el código

Ubuntu/Debian, permisos sudo, al menos 12 GB libres y 8 GB de RAM recomendados. La instalación necesita Internet para descargar dependencias y modelos; el funcionamiento posterior se plantea como local.

2 Código fuente completo

El siguiente bloque reproduce el contenido del archivo adjunto sin ejecutar ni modificar su lógica.

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""PNLIO 2031 — Despliegue unificado ADÁN 2031 + EVA 2031 (un solo archivo).

Deja en Ubuntu todo el ecosistema local PNLIO:
  1. ADÁN 2031 Chat        -> chat local con Ollama que lee PDF/Word/Excel (phi3:mini)
  2. Kernel SOLEDAD v1.3   -> memoria SQLite FTS5, voz, visión y núcleo robótico
                              supervisado (15 pruebas de seguridad)
  3. EVA 2031 v1.7.2       -> dashboard Gradio con RAG (FAISS), visión (moondream),
                              voz (pyttsx3), oído (faster-whisper) y Capa de
                              Seguridad Física (CSP): Intenciones -> CSP -> Actuadores

USO (copiar y pegar en la terminal de Ubuntu):
    nano despliegue_pnlio2031.py    -> pegar, Ctrl+O, Enter, Ctrl+X
    python3 despliegue_pnlio2031.py

Requisitos: Ubuntu/Debian, sudo, 12 GB libres en disco, 8 GB de RAM recomendados
(para 5 GB de RAM instala solo ADÁN: EVA completa con embeddings + whisper es pesada).
Internet solo durante la instalación. Los núcleos robóticos operan en SIMULACIÓN;
el hardware real está bloqueado hasta tener driver verificado.

Copyright 2026 Gonzalo Mauricio De la Rivera Arellano
SPDX-License-Identifier: Apache-2.0
"""
import os
import shutil
import subprocess
import sys
import time
import urllib.request
from pathlib import Path

BANNER = """
╔═══════════════════════════════════════════╗
║           PNLIO 2031 — FAMILIA            ║
║   ADÁN 2031 Chat + SOLEDAD Kernel v1.3    ║
║   EVA 2031 v1.7.2 · CSP · RAG · Visión    ║
║      Auto-despliegue · 100% local         ║
╚═══════════════════════════════════════════╝
"""
MODELO_TEXTO = "phi3:mini"                # 2.3 GB, fluido en i7-6700 sin GPU
```

```
MODELO_VISION = "moondream"                # ~1.8 GB, visión local para EVA y /vision
CARPETA = Path.home() / "pnlio2031"
ENTORNO = CARPETA / "env"
KERNEL_DIR = CARPETA / "kernel"
KERNEL_ARCHIVO = KERNEL_DIR / "soledad_kernel_v1_3.py"
DASH_ARCHIVO = CARPETA / "dashboard.py"
PUERTO_OLLAMA = "http://localhost:11434"
ESPACIO_MINIMO_GB = 12                         # 2 modelos 4.1 GB + venv (torch/whisper)
+ margen
RAM_AVISO_GB = 8                           # por debajo: EVA completa queda pesada

CHAT_PY = '#!/usr/bin/env python3\n# -*- coding: utf-8 -*-\n"""COMANDANTE ADÁN 2031 —
Chat local con Ollama que lee tus documentos.\n\n  /doc <ruta>  -> carga PDF, Word,
Excel o txt en la conversación\n  /docs         -> lista los documentos cargados\n  /
limpiar     -> borra documentos y conversación\n  /salir       -> terminar\nTodo corre
en localhost:11434. Nada sale de esta máquina.\n© 2026 Gonzalo Mauricio De la Rivera
Arellano — Apache 2.0\nSPDX-License-Identifier: Apache-2.0\n"""\nimport json\nfrom pathlib
import Path\n\nimport requests\nfrom pypdf import PdfReader\nimport docx\nimport pandas
as pd\n\nAPI = "http://localhost:11434/api/chat"\nMODELO = "phi3:mini"\nMAX_CARACTERES_DOC
= 10_000    # phi3:mini tiene ~3.8k tokens de contexto; 24k caracteres se truncaban
en silencio\n\ndef extraer_texto(ruta: str) -> str:\n    p = Path(ruta).expanduser()\n
if not p.exists():\n          raise FileNotFoundError(f"no existe: {p}")\n    suf =
p.suffix.lower()\n  if suf == ".pdf":\n     return "\\n".join(pg.extract_text() or "" for
pg in PdfReader(str(p)).pages)\n  if suf == ".docx":\n      d = docx.Document(str(p))\n
return "\\n".join(par.text for par in d.paragraphs if par.text)\n  if suf in (".xlsx",
".xls"):\n          return "\\n".join(h.fillna("").astype(str).to_string(index=False)\n
for h in pd.read_excel(str(p), sheet_name=None).values())\n  if suf in (".txt", ".md",
".csv"):\n          return p.read_text(encoding="utf-8", errors="ignore")\n    raise
ValueError(f"formato no soportado: {suf} (pdf, docx, xlsx, xls, txt, csv)")\n\ndef
chat(mensajes):\n  with requests.post(API, json={"model": MODELO, "messages": mensajes,
\n                                    "stream": True}, stream=True, timeout=600) as r:
\n      r.raise_for_status()\n      partes = []\n      for linea in r.iter_lines():
\n          if not linea:\n              continue\n          dato = json.loads(linea)\n
trozo = dato.get("message", {}).get("content", "")\n              print(trozo, end="",
flush=True)\n              partes.append(trozo)\n              if dato.get("done"):\n
break\n      print()\n      return "".join(partes)\n\ndef main():\n  docs, historial
= [], []\n  print("=" * 60)\n  print("  COMANDANTE ADÁN 2031 — Chat local (phi3:mini,
offline)")\n  print("  /doc <archivo>  para cargar PDF/Word/Excel/txt")\n  print("="
* 60)\n  while True:\n      try:\n          usuario = input("\\nTú > ").strip()\n
except (EOFError, KeyboardInterrupt):\n              break\n          if not usuario:\n
continue\n          if usuario == "/salir":\n              break\n          if usuario ==
"/limpiar":\n              docs.clear(); historial.clear()\n              print("(memoria
limpiada)"); continue\n          if usuario == "/docs":\n              print("\\n".join(f"
```

```
- {n} ({len(t)} car.)" for n, t in docs) or "(sin documentos)")\n            continue\n
if usuario.startswith("/doc "):\n            ruta = usuario[5:].strip().strip(\'"\')\n
try:\n                texto = extraer_texto(ruta)\n                if len(texto)
> MAX_CARACTERES_DOC:\n                            print(f"(recortado de {len(texto)} a
{MAX_CARACTERES_DOC} caracteres)")\n              texto = texto[:MAX_CARACTERES_DOC]\n
docs.append((Path(ruta).name, texto))\n            print(f"(cargado: {Path(ruta).name},
{len(texto)} caracteres)")\n          except Exception as e:\n            print(f"(error
al cargar: {e})")\n            continue\n        sistema = ("Eres ADÁN 2031, asistente
local en español, directo y útil. "\n                        "Con documentos: responde
apoyándote en ellos y cita la parte relevante.")\n        if docs:\n            sistema
+= "\\n\\nDocumentos:\\n" + "\\n\\n".join(f"### {n}\\n{t}" for n, t in docs)\n        if
not historial:\n            historial.append({"role": "system", "content": sistema})\n
else:\n          historial[0] = {"role": "system", "content": sistema}  # contexto siempre
vigente\n        historial.append({"role": "user", "content": usuario})\n        try:\n
print("\\nADÁN 2031 >", end=" ")\n          respuesta = chat([historial[0]] + historial[1:]
[-29:])\n              historial.append({"role": "assistant", "content": respuesta})\n
except requests.exceptions.ConnectionError:\n            print("(sin conexión con Ollama:
systemctl status ollama)")\n        except Exception as e:\n            print(f"(error:
{e})")\n    print("\\nComandante fuera. Todo quedó en tu máquina.")\n\nif __name__ ==
"__main__":\n    main()\n'
```

```
KERNEL_PY = '#!/usr/bin/env python3\n# -*- coding: utf-8 -*-\n# © 2026 Gonzalo Mauricio De
la Rivera Arellano — Apache 2.0\n# SPDX-License-Identifier: Apache-2.0\n#\n# Licensed under
the Apache License, Version 2.0 (the "License");\n# you may not use this file except in
compliance with the License.\n# You may obtain a copy of the License at http://www.apache.
org/licenses/LICENSE-2.0\n# Unless required by applicable law or agreed to in writing,
software\n# distributed under the License is distributed on an "AS IS" BASIS,\n# WITHOUT
WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.\n"""PNLIO SOLEDAD Kernel
v1.3 — Memoria, Percepción y Núcleo Robótico 2028\n\nAutoría: Gonzalo Mauricio De la Rivera
Arellano\n\nRequisitos:\n    pip install requests pyttsx3 opencv-python\n    Ollama local
con un modelo (ollama pull phi3:mini). Visión real: ollama pull moondream\n\nCambios v1.3
respecto a v1.2:\n    [FIX] FTS5: el relleno de notas antiguas nunca corría; ahora usa
\'rebuild\'.\n    [FIX] Consulta FTS5 robusta (extrae palabras, cada una entrecomillada,
unidas con OR).\n    [FIX] /api/chat ignoraba la clave "system" de nivel superior; ahora va
como mensaje system.\n    [FIX] pyttsx3 se crea dentro del hilo de voz (SAPI5 es sensible
al hilo).\n    [FIX] Conexiones SQLite se cierran de verdad (antes solo hacían commit).
\n    [FIX] /ver conserva el historial: la imagen viaja en el último mensaje de /api/
chat.\n    [MEJORA] Multimodal detectado vía /api/show (capacidad \'vision\') con lista
de respaldo.\n    [MEJORA] Movimiento detectado alimenta al núcleo robótico (limita la
velocidad).\n    [NUEVO] Núcleo Robótico 2028: supervisor de seguridad independiente,
PID con\n              anti-windup, simulador y comando /robot run X Y. Hardware real
BLOQUEADO\n              hasta tener driver verificado.\n    [SEGURIDAD] Operación 100%
```

```
local; ningún dato sale del dispositivo.\n"""\nfrom __future__ import annotations\n\nimport base64\nimport contextlib\nimport math\nimport os\nimport queue\nimport re\nimport sqlite3\nimport sys\nimport threading\nimport uuid\nfrom dataclasses import dataclass, field\nfrom datetime import datetime\nfrom pathlib import Path\nfrom typing import Any\n\nimport requests\n\ntry:\n    import pyttsx3\n    TTS_AVAILABLE = True\nexcept ImportError:\n    TTS_AVAILABLE = False\n\ntry:\n    import cv2\n    import numpy as np\n    VISION_AVAILABLE = True\nexcept ImportError:\n    VISION_AVAILABLE = False\n\nAPP_NAME = "PNLIO SOLEDAD Kernel + Núcleo Robótico 2028"\nVERSION = "1.3.0"\nOLLAMA_URL = os.getenv("PNLIO_OLLAMA_URL", "http://127.0.0.1:11434")\nMODEL_NAME = os.getenv("PNLIO_MODEL", "phi3")\nMODELOS_MULTIMODALES = {"moondream", "llava", "llava-phi3", "bakllava",\n                        "llama3.2-vision", "minicpm-v"}\nREQUEST_TIMEOUT = int(os.getenv("PNLIO_TIMEOUT", "120"))\nMAX_CONTEXT_MESSAGES = int(os.getenv("PNLIO_CONTEXT_MESSAGES", "12"))\nMAX_MEMORY_RESULTS = int(os.getenv("PNLIO_MEMORY_RESULTS", "5"))\nMOVIMIENTO_UMBRAL_PORCENTAJE = float(os.getenv("PNLIO_MOV_UMBRAL", "3.5"))\nFRAMES_CALENTAMIENTO = int(os.getenv("PNLIO_CAM_WARMUP", "15"))\n\nBASE_DIR = Path.home() / ".pnlio"\nDB_PATH = Path(os.getenv("PNLIO_DB_PATH", str(BASE_DIR / "soledad_memory.sqlite3")))\nIMAGE_PATH = BASE_DIR / "ultima_captura.jpg"  # RECONSTRUIDO: el PDF truncaba el nombre\n\nSYSTEM_TEMPLATE = """Eres SOLEDAD, asistente local del kernel PNLIO con capacidades sensoriales y un\nnúcleo robótico supervisado. Principios operativos: memoria, continuidad, contexto y percepción local.\nHabla en español salvo que el usuario pida otro idioma. Sé precisa, honesta y breve. No inventes datos,\nnoticias ni fechas. La fecha/hora del sistema y los datos de sensores se proporcionan en el mensaje de\ncontexto; úsalos como referencia. Si se te entrega una imagen del entorno, descríbela con precisión y\nsin especular más allá de lo visible. No afirmes haber buscado en Internet: este kernel opera 100% offline.\nNunca ordenes movimiento físico por tu cuenta: el movimiento solo lo inicia la persona con /robot run y\nsiempre pasa por el supervisor de seguridad. Prioridad absoluta: vida y seguridad de la persona."""\n\nDIAS_ES = ["lunes", "martes", "miércoles", "jueves", "viernes", "sábado", "domingo"]\nMESES_ES = ["enero", "febrero", "marzo", "abril", "mayo", "junio", "julio",\n            "agosto", "septiembre", "octubre", "noviembre", "diciembre"]\n\n\ndef fecha_local() -> str:\n    now = datetime.now().astimezone()\n    return (f"{DIAS_ES[now.weekday()]} {now.day} de {MESES_ES[now.month - 1]} de {now.year}, "\n            f"{now:%H:%M:%S} (UTC{now:%z})")\n\n\ndef preparar_directorios() -> None:\n    BASE_DIR.mkdir(parents=True, exist_ok=True)\n\n\n# ------------------------------ CAPA DE INTERFAZ ------------------------------\nclass EntradaSalida:\n    def leer(self, prompt: str) -> str:\n        return input(prompt).strip()\n\n    def escribir(self, texto: str) -> None:\n        print(texto)\n\n    def confirmar(self, pregunta: str) -> bool:\n        return self.leer(pregunta).strip().upper() == "SI"\n\n\n# ------------------------------ NÚCLEO ROBÓTICO 2028 ------------------------\n# Arquitectura: orden del planificador -> SUPERVISOR -> actuador.\n# El supervisor NO depende del planificador: si este falla, el supervisor para.\n# Estado seguro: velocidades = 0 (robot móvil terrestre). Los paros se ENGANCHAN\n# y solo se liberan con rearmar() explícito. Unidades SI: m, s, rad. Reloj: simulado.\n\ndef _envolver_angulo(a: float) -> float:\n    return math.atan2(math.sin(a),
```

```
math.cos(a))\n\n\ndef _limitar(x: float, lo: float, hi: float) -> float:\n    return max(lo, min(hi, x))\n\n\n@dataclass\nclass LimitesRobot:\n    v_max: float = 0.5          # m/s\n    w_max: float = 1.0          # rad/s\n    a_max: float = 0.5          # m/s^2 (aceleración)\n    alfa_max: float = 2.0       # rad/s^2\n    freno_factor: float = 2.0   # frenar es hasta 2x más rápido que acelerar\n    timeout_orden: float = 0.5  # s sin órdenes nuevas -> paro\n    timeout_sensor: float = 0.5 # s sin dato fresco del sensor -> paro\n    d_parada: float = 0.40      # m: obstáculo más cerca -> paro enganchado\n    d_lento: float = 1.20       # m: zona de reducción lineal de velocidad\n    v_cerca_persona: float = 0.15 # m/s si hay movimiento sospechoso en cámara\n    v_aproximacion: float = 0.05 # m/s mínima en zona lenta (evita quedarse \'colgado\')\n\n\nclass SupervisorSeguridad:\n    """Última línea de software. NO sustituye el corte de energía por hardware."""\n\n    def __init__(self, limites: LimitesRobot | None = None):\n        self.lim = limites or LimitesRobot()\n        self.paro = False\n        self.razon_paro = ""\n        self._v = 0.0\n        self._w = 0.0\n\n    def declarar_paro(self, razon: str) -> None:\n        if not self.paro:\n            self.paro, self.razon_paro = True, razon\n        self._v = self._w = 0.0\n\n    def rearmar(self) -> None:\n        """Acción humana explícita. Nunca se llamar automáticamente."""\n        self.paro, self.razon_paro = False, ""\n        self._v = self._w = 0.0\n\n    def filtrar(self, v_cmd: float, w_cmd: float, t_cmd: float, t_sensor: float,\n                distancia: float, ahora: float, dt: float,\n                persona_cerca: bool = False) -> tuple[float, float]:\n        L = self.lim\n        if self.paro:\n            return 0.0, 0.0\n        if dt <= 0 or not math.isfinite(dt):\n            self.declarar_paro("dt inválido")\n            return 0.0, 0.0\n        if not all(math.isfinite(x) for x in (v_cmd, w_cmd)):\n            self.declarar_paro("orden no finita (NaN/inf)")\n            return 0.0, 0.0\n        if ahora - t_cmd > L.timeout_orden:\n            self.declarar_paro("orden caducada (planificador colgado)")\n            return 0.0, 0.0\n        if ahora - t_sensor > L.timeout_sensor or math.isnan(distancia):\n            # inf = nada detectado (válido)\n            self.declarar_paro("sensor de distancia caducado o inválido")\n            return 0.0, 0.0\n        if distancia <= L.d_parada:\n            self.declarar_paro(f"obstáculo a {distancia:.2f} m")\n            return 0.0, 0.0\n        v_tope = L.v_max\n        if persona_cerca:\n            v_tope = min(v_tope, L.v_cerca_persona)\n        if distancia < L.d_lento:\n            frac = (distancia - L.d_parada) / (L.d_lento - L.d_parada)\n            v_tope = min(v_tope, max(L.v_aproximacion, L.v_max * _limitar(frac, 0.0, 1.0)))\n        v = _limitar(v_cmd, 0.0, v_tope)          # sin marcha atrás en este perfil\n        w = _limitar(w_cmd, -L.w_max, L.w_max)\n        dv_up = L.a_max * dt\n        dv_down = L.a_max * L.freno_factor * dt\n        v = _limitar(v, self._v - dv_down, self._v + dv_up)\n        dw = L.alfa_max * dt\n        w = _limitar(w, self._w - dw, self._w + dw)\n        self._v, self._w = v, w\n        return v, w\n\n\nclass PIDAntiWindup:\n    """PID con saturación, anti-windup (integración condicional) y derivada filtrada."""\n\n    def __init__(self, kp: float, ki: float, kd: float, lim_salida: float, tau_d: float = 0.05):\n        self.kp, self.ki, self.kd = kp, ki, kd\n        self.lim, self.tau = lim_salida, tau_d\n        self.integral = 0.0\n        self._medida_prev: float | None = None\n        self._deriv = 0.0\n\n    def reset(self) -> None:\n        self.integral, self._medida_prev, self._deriv = 0.0, None, 0.0\n\n    def paso(self, error: float, medida: float, dt: float) -> float:\n        if
```

```
dt <= 0 or not math.isfinite(dt):\n                raise ValueError("dt debe ser > 0")\n
if self._medida_prev is not None:\n                crudo = -(medida - self._medida_prev) /
dt\n                a = dt / (self.tau + dt)\n                self._deriv += a * (crudo -
self._deriv)\n            self._medida_prev = medida\n            sin_sat = self.kp * error +
self.ki * self.integral + self.kd * self._deriv\n          sat = _limitar(sin_sat, -self.lim,
self.lim)\n          if sat == sin_sat or (error * sin_sat < 0):   # integra solo si no empuja
la saturación\n                self.integral += error * dt\n          return _limitar(self.kp
* error + self.ki * self.integral + self.kd * self._deriv,\n                                  -
self.lim, self.lim)\n\n\n@dataclass\nclass Obstaculo:\n    x: float\n    y: float\n    r:
float\n\n\nclass RobotSimulado:\n    """Monociclo 2D con sensor de distancia frontal.
Permite inyectar fallas del sensor."""\n\n    def __init__(self, obstaculos: list[Obstaculo]
| None = None):\n            self.x = self.y = self.th = 0.0\n            self.obstaculos =
obstaculos or []\n            self.sensor_congelado = False\n            self._ultimo_valor =
float("inf")\n            self.t_sensor = 0.0\n\n    def leer_distancia(self, t: float) ->
tuple[float, float]:\n            if self.sensor_congelado:          # dato congelado: el sello
de tiempo NO avanza\n                return self._ultimo_valor, self.t_sensor\n            mejor
= float("inf")\n            for o in self.obstaculos:\n                dx, dy = o.x - self.x,
o.y - self.y\n                ang = abs(_envolver_angulo(math.atan2(dy, dx) - self.th))\n
if ang <= 0.35:\n                    mejor = min(mejor, math.hypot(dx, dy) - o.r)\n
self._ultimo_valor, self.t_sensor = mejor, t\n          return mejor, t\n\n    def aplicar(self,
v: float, w: float, dt: float) -> None:\n            self.th = _envolver_angulo(self.th + w *
dt)\n            self.x += v * math.cos(self.th) * dt\n            self.y += v * math.sin(self.th)
* dt\n\n\n@dataclass\nclass ResultadoRun:\n    exito: bool\n    motivo: str\n    tiempo:
float\n    distancia_final: float\n    pasos: int\n    trayectoria: list[tuple[float,
float]] = field(default_factory=list)\n\n\nclass NucleoRobotico:\n    DT = 0.05  # 20
Hz\n\n    def __init__(self, limites: LimitesRobot | None = None):\n            self.supervisor
= SupervisorSeguridad(limites)\n            self.persona_cerca = False    # lo alimenta la
detección de movimiento de la cámara\n            self.modo = "SIMULACION"\n            self.ultimo:
ResultadoRun | None = None\n\n    def paro_emergencia(self, razon: str = "paro manual") -
> None:\n            self.supervisor.declarar_paro(razon)\n\n    def rearmar(self) -> None:\n
self.supervisor.rearmar()\n\n    def estado(self) -> str:\n            s = self.supervisor\n
paro = f"PARO ENGANCHADO ({s.razon_paro})" if s.paro else "armado"\n            ult = ""\n
if self.ultimo:\n                ult = (f" | último run: {\'OK\' if self.ultimo.exito
else \'FALLÓ\'} — {self.ultimo.motivo}"\n                       f" ({self.ultimo.pasos}
pasos, {self.ultimo.tiempo:.1f}s)")\n            return f"Modo {self.modo} | supervisor {paro}
{ult}"\n\n    def run(self, objetivo: tuple[float, float], robot: RobotSimulado | None =
None,\n            t_max: float = 90.0, tolerancia: float = 0.10,\n            falla_orden_en:
float | None = None) -> ResultadoRun:\n            """Lleva el robot simulado hasta `objetivo`
(metros) pasando SIEMPRE por el supervisor.\n            `falla_orden_en`: segundo en que el
planificador \'se cuelga\' (inyección de falla)."""\n            if self.modo != "SIMULACION":\n
raise RuntimeError("Hardware real bloqueado: no hay driver verificado.")\n            if
self.supervisor.paro:\n                return ResultadoRun(False, f"rechazado: paro enganchado
({self.supervisor.razon_paro})",\n                                    0.0, float("nan"), 0)\n
```

```
if not all(math.isfinite(c) for c in objetivo):\n            return ResultadoRun(False,
"objetivo no finito", 0.0, float("nan"), 0)\n        robot = robot or RobotSimulado()\n
pid_giro = PIDAntiWindup(kp=2.0, ki=0.05, kd=0.1, lim_salida=1.0)  # RECONSTRUIDO: PDF
truncaba lim_salida\n       t, pasos = 0.0, 0\n       t_cmd = 0.0\n       v_cmd = w_cmd =
0.0\n       tray: list[tuple[float, float]] = []\n       gx, gy = objetivo\n       while
t < t_max:\n          dist = math.hypot(gx - robot.x, gy - robot.y)\n          if dist <=
tolerancia:\n              res = ResultadoRun(True, "objetivo alcanzado", t, dist, pasos,
tray)\n            self.ultimo = res\n            return res\n         # ---- planificador
(puede fallar; el supervisor no depende de él) ----\n          if falla_orden_en is None or
t < falla_orden_en:\n               err_th = _envolver_angulo(math.atan2(gy - robot.y, gx
- robot.x) - robot.th)\n               w_cmd = pid_giro.paso(err_th, robot.th, self.DT)\n
v_cmd = _limitar(0.8 * dist, 0.0, self.supervisor.lim.v_max) * max(0.0, math.cos(err_th))\n
t_cmd = t\n               d_frente, t_sens = robot.leer_distancia(t)\n               v,
w = self.supervisor.filtrar(v_cmd, w_cmd, t_cmd, t_sens, d_frente, t, self.DT,\n
persona_cerca=self.persona_cerca)\n          if self.supervisor.paro:\n              res
= ResultadoRun(False, f"PARO: {self.supervisor.razon_paro}", t, dist, pasos, tray)\n
self.ultimo = res\n                    return res\n               robot.aplicar(v, w,
self.DT)\n           t += self.DT\n           pasos += 1\n           if pasos % 10 == 0:\n
tray.append((robot.x, robot.y))\n       res = ResultadoRun(False, "tiempo máximo agotado",
t, dist, pasos, tray)\n      self.ultimo = res\n      return res\n\n\ndef autoprueba_robot()
-> bool:\n    """Pruebas de diseño del núcleo. Cada una puede fallar. Resultado: lista de
OK/FALLA."""\n    resultados: list[tuple[str, bool]] = []\n\n    def chk(nombre: str, cond:
bool) -> None:\n       resultados.append((nombre, bool(cond)))\n\n    # 1) NaN rechazado y
paro enganchado\n    s = SupervisorSeguridad()\n    s.filtrar(float("nan"), 0.0, 0.0, 0.0,
5.0, 0.0, 0.05)\n    chk("NaN provoca paro", s.paro)\n    v, w = s.filtrar(0.3, 0.0, 0.1,
0.1, 5.0, 0.1, 0.05)\n    chk("paro enganchado: ignora órdenes buenas", (v, w) == (0.0,
0.0))\n    s.rearmar()\n    chk("rearmar() libera el paro", not s.paro)\n\n    # 2) límite
de aceleración\n    s = SupervisorSeguridad()\n    v, _ = s.filtrar(0.5, 0.0, 0.0, 0.0, 5.0,
0.0, 0.05)\n    chk("aceleración limitada (a_max*dt)", abs(v - 0.025) < 1e-9)\n\n    # 3)
orden caducada\n    s = SupervisorSeguridad()\n    s.filtrar(0.3, 0.0, 0.0, 1.0, 5.0, 1.0,
0.05)\n    chk("orden caducada provoca paro", s.paro)\n\n    # 4) sensor caducado (dato
congelado)\n    s = SupervisorSeguridad()\n    s.filtrar(0.3, 0.0, 1.0, 0.0, 5.0, 1.0,
0.05)\n   chk("sensor caducado provoca paro", s.paro)\n\n   # 5) obstáculo cercano\n   s =
SupervisorSeguridad()\n   s.filtrar(0.3, 0.0, 0.0, 0.0, 0.2, 0.0, 0.05)\n   chk("obstáculo
<d_parada provoca paro", s.paro)\n\n   # 6) anti-windup: tras bloqueo largo la integral no
se dispara\n   pid = PIDAntiWindup(1.0, 1.0, 0.0, lim_salida=1.0)\n   for _ in range(200):
\n       pid.paso(10.0, 0.0, 0.05)\n    chk("anti-windup acota la integral", pid.integral
< 3.0)\n    try:\n         pid.paso(1.0, 0.0, 0.0)\n         chk("PID rechaza dt <= 0",
False)\n    except ValueError:\n       chk("PID rechaza dt <= 0", True)\n\n    # 7) misión
feliz\n    n = NucleoRobotico()\n    r = n.run((3.0, 1.0))\n    chk("run alcanza el objetivo
en simulación", r.exito)\n\n    # 8) obstáculo en el camino -> paro (no choca)\n    n =
NucleoRobotico()\n    r = n.run((4.0, 0.0), RobotSimulado([Obstaculo(2.0, 0.0, 0.2)]))\n
chk("run se detiene ante obstáculo", (not r.exito) and "obstáculo" in r.motivo)\n\n    #
```

```
9) planificador colgado -> paro\n     n = NucleoRobotico()\n     r = n.run((6.0, 0.0),
falla_orden_en=1.0)\n      chk("planificador colgado provoca paro", (not r.exito) and
"caducada" in r.motivo)\n\n    # 10) sensor congelado -> paro\n    n = NucleoRobotico()\n
rob = RobotSimulado()\n     rob.leer_distancia(0.0)\n     rob.sensor_congelado = True\n
r = n.run((3.0, 0.0), rob)\n      chk("sensor congelado provoca paro", (not r.exito)
and "sensor" in r.motivo)\n\n     # 11) tras un paro, un nuevo run se rechaza hasta
rearmar\n     r2 = n.run((3.0, 0.0))\n     chk("run rechazado con paro enganchado", (not
r2.exito) and "rechazado" in r2.motivo)\n\n     # 12) persona cerca reduce velocidad\n
n1, n2 = NucleoRobotico(), NucleoRobotico()\n      n2.persona_cerca = True\n      t1
= n1.run((3.0, 0.0)).tiempo\n      t2 = n2.run((3.0, 0.0)).tiempo\n      chk("persona
cerca = más lento", t2 > t1)\n\n      ok = all(c for _, c in resultados)\n      for
nombre, c in resultados:\n          print(("OK     " if c else "FALLA ") + nombre)\n
print(f"Autoprueba núcleo robótico: {sum(c for _, c in resultados)}/{len(resultados)}")\n
return ok\n\n\nnucleo = NucleoRobotico()\n\n#  ------------------------------ MOTOR
SENSORIAL ------------------------------\nclass MotorSensorial:\n    def __init__(self):
\n         self.voz_activa = TTS_AVAILABLE\n         self._cola_voz: "queue.Queue[str |
None]" = queue.Queue()\n        self._fotograma_previo = None\n        if TTS_AVAILABLE:\n
threading.Thread(target=self._bucle_voz, daemon=True).start()\n\n    def _bucle_voz(self)
-> None:\n        # El motor se crea Y se usa en este hilo (SAPI5 es sensible al hilo)\n
try:\n             motor = pyttsx3.init()\n             motor.setProperty("rate", 165)\n
except Exception:\n          self.voz_activa = False\n          return\n      while True:\n
try:\n                 texto = self._cola_voz.get()\n                 if texto is None:\n
break\n             motor.say(texto)\n             motor.runAndWait()\n          except
Exception:\n           pass\n        finally:\n           self._cola_voz.task_done()\n\n
def hablar(self, texto: str) -> None:\n         if TTS_AVAILABLE and self.voz_activa:\n
self._cola_voz.put(texto)\n\n    def esperar_voz(self) -> None:\n        if TTS_AVAILABLE
and self.voz_activa:\n         self._cola_voz.join()\n\n   def _capturar_fotograma(self):
\n       captura = cv2.VideoCapture(0)\n       try:\n          if not captura.isOpened():
\n                 return None\n             for _ in range(FRAMES_CALENTAMIENTO):   #
deja estabilizar autoexposición\n                    captura.read()\n              ret,
frame = captura.read()\n              return frame if ret else None\n          finally:
\n          captura.release()\n\n    def capturar_vision(self) -> tuple[str, bytes | None,
bool]:\n       if not VISION_AVAILABLE:\n           return "Módulo de visión no disponible
(falta OpenCV o hardware de cámara).", None, False\n         try:\n             frame =
self._capturar_fotograma()\n         if frame is None:\n            return "Error: la cámara
web no pudo capturar ningún fotograma.", None, False\n           preparar_directorios()\n
ok, buf = cv2.imencode(".jpg", frame, [cv2.IMWRITE_JPEG_QUALITY, 85])\n              if
not ok:\n                    return "Error al codificar el fotograma.", None, False\n
jpeg = buf.tobytes()\n               IMAGE_PATH.write_bytes(jpeg)\n               mov =
self._detectar_movimiento(frame)\n              msg = f"Imagen del entorno capturada en
{IMAGE_PATH}."\n             if mov:\n                 msg += " ADVERTENCIA: movimiento
significativo en la escena (el robot limitará su velocidad)."\n          return msg, jpeg,
mov\n        except Exception as exc:\n            return f"Error en la captura visual:
```

```
{exc}", None, False\n\n    def _detectar_movimiento(self, frame) -> bool:\n        previo, self._fotograma_previo = self._fotograma_previo, frame\n            if previo is None:\n return False\n            try:\n                g1 = cv2.GaussianBlur(cv2.cvtColor(previo, cv2.COLOR_BGR2GRAY), (21, 21), 0)\n                g2 = cv2.GaussianBlur(cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY), (21, 21), 0)\n            _, umbral = cv2.threshold(cv2.absdiff(g1, g2), 25, 255, cv2.THRESH_BINARY)\n            return float(np.count_nonzero(umbral)) * 100.0 / umbral.size >= MOVIMIENTO_UMBRAL_PORCENTAJE\n        except Exception:\n            return False\n\n\nsensores = MotorSensorial()\n\n# ---------------------------------- MEMORIA ----------------------------------\n@contextlib.contextmanager\ndef conectar_db():\n """Abre, confirma (o revierte) y CIERRA la conexión."""\n    preparar_directorios()\n conn = sqlite3.connect(DB_PATH, timeout=10)\n    try:\n            conn.execute("PRAGMA journal_mode=WAL")\n        conn.execute("PRAGMA foreign_keys=ON")\n        yield conn\n conn.commit()\n    except Exception:\n        conn.rollback()\n        raise\n    finally:\n conn.close()\n\n\ndef inicializar_memoria() -> None:\n    with conectar_db() as conn:\n conn.execute("""CREATE TABLE IF NOT EXISTS conversations (\n        id TEXT PRIMARY KEY,\n created_at TEXT NOT NULL,\n        title TEXT NOT NULL)""")\n    conn.execute("""CREATE TABLE IF NOT EXISTS messages (\n                id INTEGER PRIMARY KEY AUTOINCREMENT,\n conversation_id TEXT NOT NULL,\n            role TEXT NOT NULL CHECK(role IN (\'user\', \'assistant\')),\n                content TEXT NOT NULL,\n                created_at TEXT NOT NULL,\n            FOREIGN KEY(conversation_id) REFERENCES conversations(id))""")\n conn.execute("""CREATE TABLE IF NOT EXISTS memory_notes (\n        id INTEGER PRIMARY KEY AUTOINCREMENT,\n            note TEXT NOT NULL,\n            created_at TEXT NOT NULL)""")\n try:\n            existia = conn.execute(\n                "SELECT 1 FROM sqlite_master WHERE name=\'memory_notes_fts\'").fetchone()\n                    conn.execute("""CREATE VIRTUAL TABLE IF NOT EXISTS memory_notes_fts\n                        USING fts5(note, content=\'memory_notes\', content_rowid=\'id\')""")\n            conn.execute("""CREATE TRIGGER IF NOT EXISTS trg_notas_ai AFTER INSERT ON memory_notes\n                BEGIN INSERT INTO memory_notes_fts(rowid, note) VALUES (new.id, new.note); END""")\n conn.execute("""CREATE TRIGGER IF NOT EXISTS trg_notas_ad AFTER DELETE ON memory_notes\n BEGIN INSERT INTO memory_notes_fts(memory_notes_fts, rowid, note)\n                VALUES (\'delete\', old.id, old.note); END""")\n            conn.execute("""CREATE TRIGGER IF NOT EXISTS trg_notas_au AFTER UPDATE ON memory_notes\n                BEGIN INSERT INTO memory_notes_fts(memory_notes_fts, rowid, note)\n            VALUES (\'delete\', old.id, old.note);\n                    INSERT INTO memory_notes_fts(rowid, note) VALUES (new.id, new.note); END""")\n            if not existia:   # índice nuevo sobre notas antiguas: reconstruir\n                conn.execute("INSERT INTO memory_notes_fts(memory_notes_fts) VALUES(\'rebuild\')")\n        except sqlite3.OperationalError:\n                pass  # SQLite sin FTS5: se usa el respaldo LIKE\n                conn.execute("CREATE INDEX IF NOT EXISTS idx_messages_conv_id ON messages(conversation_id, id)")\n\n\ndef crear_conversacion() -> str:\n    cid = str(uuid.uuid4())\n    with conectar_db() as conn:\n      conn.execute("INSERT INTO conversations(id, created_at, title) VALUES (?, ?, ?)",\n (cid, datetime.now().astimezone().isoformat(), "Sesión SOLEDAD"))\n    return cid\n\n\ndef guardar_mensaje(conversation_id: str, role: str, content: str) -> None:\n        with
```

```
conectar_db() as conn:\n        conn.execute("INSERT INTO messages(conversation_id, role,
content, created_at) VALUES (?, ?, ?, ?)",\n                (conversation_id, role, content,
datetime.now().astimezone().isoformat()))\n\n\ndef  leer_historial(conversation_id:  str,
limit: int = MAX_CONTEXT_MESSAGES) -> list[dict[str, str]]:\n     with conectar_db() as
conn:\n         rows = conn.execute(\n              "SELECT role, content FROM messages
WHERE conversation_id = ? ORDER BY id DESC LIMIT ?",\n                (conversation_id,
limit)).fetchall()\n         rows.reverse()\n         return [{"role": r, "content": c}
for r, c in rows]\n\n\ndef guardar_nota(texto: str) -> int:\n    texto = texto.strip()\n
if not texto:\n           raise ValueError("La nota no puede estar vacía.")\n     with
conectar_db() as conn:\n            cur = conn.execute("INSERT INTO memory_notes(note,
created_at) VALUES (?, ?)",\n                                                    (texto,
datetime.now().astimezone().isoformat()))\n                return cur.lastrowid\n\n\ndef
_like_escape(palabra:  str)  ->  str:\n         return  palabra.replace("\\\\",  "\\\\\\
\\").replace("%",  "\\\\%").replace("_",  "\\\\_")\n\n\ndef  buscar_notas(consulta:  str,
limit: int = MAX_MEMORY_RESULTS) -> list[str]:\n    palabras = [p for p in re.findall(r"\
\w+", consulta.lower()) if len(p) > 2][:8]\n      if not palabras:\n            return
[]\n      with conectar_db() as conn:\n            try:\n                  expr = " OR
".join(f\'"{p}"\' for p in palabras)   # \\w+ no contiene comillas\n              rows =
conn.execute(\n               """SELECT n.note FROM memory_notes_fts f JOIN memory_notes n
ON n.id = f.rowid\n                WHERE memory_notes_fts MATCH ? ORDER BY rank LIMIT ?""",
\n                (expr, limit)).fetchall()\n            if rows:\n                return
[str(r[0]) for r in rows]\n         except sqlite3.OperationalError:\n              pass\n
encontrados: dict[int, str] = {}\n          for p in palabras:\n                rows =
conn.execute(\n                "SELECT id, note FROM memory_notes WHERE lower(note) LIKE ?
ESCAPE \'\\\\\'"\n           " ORDER BY id DESC LIMIT ?",\n           ("%" + _like_escape(p)
+ "%", limit)).fetchall()\n          for i, n in rows:\n              encontrados[int(i)]
= str(n)\n           return [encontrados[k] for k in sorted(encontrados, reverse=True)
[:limit]]\n\n\ndef borrar_historial(conversation_id: str) -> int:\n    with conectar_db()
as conn:\n         return conn.execute("DELETE FROM messages WHERE conversation_id = ?",
\n                       (conversation_id,)).rowcount\n\n# ------------------------------
MOTOR LOCAL --------------------------------\ndef verificar_ollama() -> tuple[bool, str]:
\n     try:\n          r = requests.get(f"{OLLAMA_URL}/api/tags", timeout=5)\n          if
not r.ok:\n               return False, f"Ollama respondió con HTTP {r.status_code}."\n
modelos = [m.get("name", "") for m in r.json().get("models", [])]\n       if not modelos:\n
return True, "Ollama responde, pero no hay modelos descargados."\n          coincide =
any(m == MODEL_NAME or m.startswith(MODEL_NAME + ":") for m in modelos)\n        detalle
= ", ".join(modelos[:8])\n          if coincide:\n                return True, f"Ollama
disponible. Modelos: {detalle}"\n       return True, f"Ollama disponible, pero no encuentro
\'{MODEL_NAME}\'. Modelos: {detalle}"\n      except requests.RequestException as exc:\n
return False, f"No se puede conectar con Ollama en {OLLAMA_URL}: {exc}"\n\n\n_cache_vision:
dict[str, bool] = {}\n\n\ndef modelo_es_multimodal(modelo: str) -> bool:\n     if modelo
in _cache_vision:\n          return _cache_vision[modelo]\n     resultado: bool | None =
None\n    try:\n       r = requests.post(f"{OLLAMA_URL}/api/show", json={"model": modelo},
```

```
timeout=5)\n       if r.ok:\n           caps = r.json().get("capabilities")\n           if
isinstance(caps, list):\n                       resultado = "vision" in caps\n     except
requests.RequestException:\n           pass\n     if resultado is None:        # Ollama
antiguo o sin respuesta: lista de respaldo\n            resultado = modelo.split(":")
[0].lower() in MODELOS_MULTIMODALES\n     _cache_vision[modelo] = resultado\n     return
resultado\n\n\ndef generar_respuesta(mensajes: list[dict[str, Any]], notas: list[str],\n
imagen_jpeg: bytes | None = None) -> str:\n   contexto = "\\n".join(f"- {n}" for n in notas)
if notas else "(sin notas relevantes)"\n    system = (SYSTEM_TEMPLATE\n              + f"\
\nFecha/hora local del sistema: {fecha_local()}.\\n"\n           + f"Sensores: voz={\'sí\'
if sensores.voz_activa else \'no\'}, "\n              f"visión={\'sí\' if VISION_AVAILABLE
else \'no\'}.\\n"\n            + f"Núcleo robótico: {nucleo.estado()}.\\n"\n            +
"Notas recuperadas de memoria persistente:\\n" + contexto)\n   msgs: list[dict[str, Any]] =
[{"role": "system", "content": system}] + [dict(m) for m in mensajes]\n   if imagen_jpeg and
modelo_es_multimodal(MODEL_NAME):\n       for m in reversed(msgs):\n           if m["role"]
== "user":\n               m["images"] = [base64.b64encode(imagen_jpeg).decode("ascii")]\n
break\n       payload = {"model": MODEL_NAME, "stream": False, "messages": msgs,\n
"options": {"temperature": 0.3}}   # RECONSTRUIDO: el PDF truncaba el valor\n     try:\n
r = requests.post(f"{OLLAMA_URL}/api/chat", json=payload, timeout=REQUEST_TIMEOUT)\n
r.raise_for_status()\n           contenido = (r.json().get("message", {}).get("content")
or "").strip()\n           return contenido or "El modelo local devolvió una respuesta
vacía."\n      except requests.Timeout:\n            return f"Tiempo de espera agotado
después de {REQUEST_TIMEOUT} s."\n     except requests.ConnectionError:\n          return
"No se pudo conectar con Ollama. Comprueba que el servicio local esté activo."\n
except Exception as exc:\n            return f"Error al procesar la respuesta local:
{exc}"\n\n# ------------------------------ TERMINAL -----------------------------------
\ndef imprimir_ayuda() -> None:\n    print("""\nComandos del Kernel SOLEDAD v1.3:\n    /
ayuda                 Mostrar esta ayuda\n    /estado                Comprobar Ollama y
modelos\n   /fecha              Fecha/hora del sistema\n   /voz on|off          Síntesis
de voz\n   /vision             Capturar fotograma y describirlo (modelo multimodal)\n   /
ver TEXTO           Preguntar sobre lo que ve la cámara\n    /modelo NOMBRE       Cambiar
modelo en esta sesión\n    /recordar TEXTO       Guardar nota (SQLite FTS5)\n    /memoria
TEXTO         Buscar notas\n    /historial             Mensajes recientes\n    /limpiar
Borrar mensajes de la sesión\n    --- Núcleo Robótico 2028 (SIMULACIÓN) ---\n    /robot
estado        Estado del supervisor\n    /robot run X Y       Llevar el robot simulado al
punto (X, Y) en metros\n   /robot paro         Paro de emergencia (enganchado)\n   /robot
rearmar       Liberar el paro (pide confirmación)\n    /robot autoprueba     Ejecutar las
pruebas del núcleo\n   /salir             Cerrar\n""")\n\n\ndef comando_robot(argumentos:
list[str], io: EntradaSalida) -> None:\n     sub = argumentos[0].lower() if argumentos
else "estado"\n     if sub == "estado":\n           print("ROBOT:", nucleo.estado())\n
elif sub == "paro":\n            nucleo.paro_emergencia("paro manual del operador")\n
print("ROBOT: PARO enganchado. Usa /robot rearmar para liberarlo.")\n     elif sub ==
"rearmar":\n         if io.confirmar("¿Confirmas que el área está despejada? Escribe SI:
"):\n             nucleo.rearmar()\n             print("ROBOT: supervisor rearmado.")\n
```

```
elif sub == "autoprueba":\n        autoprueba_robot()\n    elif sub == "run":\n        try:
\n                x, y = float(argumentos[1]), float(argumentos[2])\n            except
(IndexError, ValueError):\n                print("Uso: /robot run X Y (metros, ej: /robot
run 3 1.5)")\n                return\n            r = nucleo.run((x, y))\n            marca
= "OK" if r.exito else "DETENIDO"\n            print(f"ROBOT [{nucleo.modo}] {marca}:
{r.motivo} | t={r.tiempo:.1f}s, "\n                f"distancia final={r.distancia_final:.2f}
m, pasos={r.pasos}")\n            sensores.hablar(f"Run {\'completado\' if r.exito else
\'detenido\'}: {r.motivo}")\n        else:\n            print("Subcomandos: estado | run
X Y | paro | rearmar | autoprueba")\n\n\ndef bucle_principal() -> None:\n    global
MODEL_NAME\n    inicializar_memoria()\n    conversation_id = crear_conversacion()\n    io =
EntradaSalida()\n    print("=" * 68)\n    print(f"{APP_NAME} v{VERSION}")\n    print(f"Voz
(TTS): {\'ACTIVA\' if TTS_AVAILABLE else \'NO DISPONIBLE\'} | "\n        f"Visión (OpenCV):
{\'ACTIVA\' if VISION_AVAILABLE else \'NO DISPONIBLE\'}")\n    print(f"Núcleo robótico:
{nucleo.estado()}")\n    print(f"Memoria: {DB_PATH}")\n    print("Escribe /ayuda para ver
los comandos.")\n    print("=" * 68)\n    while True:\n        try:\n            entrada
= io.leer("\\nComandante > ")\n            if not entrada:\n                continue\n
bajo = entrada.lower()\n            partes = bajo.split()\n            if bajo in {"/salir",
"salir", "exit", "quit"}:\n                sensores.esperar_voz()\n                print("SOLEDAD:
Sesión cerrada. La memoria persistente queda guardada.")\n                break\n            if
bajo == "/ayuda":\n                imprimir_ayuda()\n                continue\n            if
bajo == "/fecha":\n                print("SOLEDAD:", fecha_local())\n                continue\n
if bajo == "/estado":\n                ok, info = verificar_ollama()\n                print(("OK:
" if ok else "ERROR: ") + info)\n                continue\n            if partes[0].lower()
== "/robot":\n                comando_robot(partes[1:], io)\n                continue\n
if bajo.startswith("/voz "):\n                arg = partes[1].lower()\n                if arg
in {"on", "off"}:\n                    sensores.voz_activa = (arg == "on") and TTS_AVAILABLE\n
print("Síntesis de voz " + ("activada." if sensores.voz_activa else "desactivada."))\n
else:\n                    print("Uso: /voz on o /voz off")\n                continue\n
if bajo == "/vision":\n                    mensaje, jpeg, mov = sensores.capturar_vision()\n
nucleo.persona_cerca = mov\n                print("SOLEDAD >", mensaje)\n                if jpeg
is not None:\n                    if modelo_es_multimodal(MODEL_NAME):\n                        desc
= generar_respuesta(\n                            [{"role": "user", "content":
"Describe brevemente el entorno que ves."}],\n                            [],
imagen_jpeg=jpeg)\n                        guardar_mensaje(conversation_id, "assistant",
f"[Visión] {desc}")\n                        print(f"SOLEDAD (visión) > {desc}")\n
sensores.hablar(desc)\n                    else:\n                        print("(Para
visión real: ollama pull moondream y /modelo moondream)")\n                continue\n
if bajo.startswith("/ver "):\n                    pregunta = entrada.split(maxsplit=1)[1]\n
mensaje, jpeg, mov = sensores.capturar_vision()\n                    nucleo.persona_cerca
= mov\n                    if jpeg is None:\n                        print("SOLEDAD >",
mensaje)\n                    continue\n                if not modelo_es_multimodal(MODEL_NAME):
\n                    print("SOLEDAD: /ver requiere un modelo multimodal (ej: /modelo
moondream).")\n                    continue\n                guardar_mensaje(conversation_id,
```

```
"user", pregunta)\n            resp = generar_respuesta(leer_historial(conversation_id),
buscar_notas(pregunta),\n                                                imagen_jpeg=jpeg)\n
guardar_mensaje(conversation_id, "assistant", resp)\n              print(f"\\nSOLEDAD >
{resp}")\n              sensores.hablar(resp)\n              continue\n          if
bajo.startswith("/modelo "):\n            nuevo = entrada.split(maxsplit=1)[1].strip()\n
if nuevo:\n                MODEL_NAME = nuevo\n                extra = " (multimodal:
visión activa)" if modelo_es_multimodal(nuevo) else ""\n                print(f"Modelo
cambiado a: {MODEL_NAME}{extra}")\n            continue\n         if bajo.startswith("/
recordar "):\n                    try:\n                        print(f"Nota guardada
(id={guardar_nota(entrada.split(maxsplit=1)[1])}).")\n                except ValueError
as exc:\n                    print(exc)\n                continue\n            if
bajo.startswith("/memoria "):\n            notas = buscar_notas(entrada.split(maxsplit=1)
[1])\n          if notas:\n              print("Notas encontradas (por relevancia):")\n
for i, n in enumerate(notas, 1):\n                  print(f"{i}. {n}")\n          else:
\n                print("No encontré notas que coincidan.")\n              continue\n
if bajo == "/historial":\n                  for m in leer_historial(conversation_id):
\n                            print(f"{\'Comandante\' if m[\'role\'] == \'user\' else
\'SOLEDAD\'}: {m[\'content\']}")\n                continue\n            if bajo == "/
limpiar":\n                if io.confirmar("¿Borrar historial de esta sesión? Escribe
SI: "):\n                        print(f"Borrados {borrar_historial(conversation_id)}
mensajes.")\n                continue\n              guardar_mensaje(conversation_id,
"user", entrada)\n            resp = generar_respuesta(leer_historial(conversation_id),
buscar_notas(entrada))\n          guardar_mensaje(conversation_id, "assistant", resp)\n
print(f"\\nSOLEDAD > {resp}")\n                sensores.hablar(resp)\n          except
(KeyboardInterrupt, EOFError):\n                print("\\nSOLEDAD: Sesión interrumpida.
Cerrando...")\n            sensores.esperar_voz()\n            break\n        except
Exception as exc:\n            print(f"\\nERROR: {type(exc).__name__}: {exc}")\n\n\nif
__name__ == "__main__":\n    if "--autoprueba-robot" in sys.argv:\n        sys.exit(0 if
autoprueba_robot() else 1)\n    bucle_principal()\n'


EVA_PY = 'import shutil\nimport gradio as gr, requests, json, base64, threading, tempfile,
time, queue\nfrom pathlib import Path\nfrom pypdf import PdfReader\nimport docx, pandas
as pd, numpy as np, faiss\nfrom sentence_transformers import SentenceTransformer\nfrom
PIL import Image\n\nVERSION = "1.7.2-EVA-KERNEL7.11"\n\n# ============ KERNEL v7.11
============\n# Contrato: los módulos emiten Intenciones; solo la CSP ejecuta.\n\nclass
Intencion:\n    def __init__(self, modulo, tipo, payload, urgencia):\n        self.modulo
= modulo      # ej: "pnlio.dialogo.pnl_inversa"\n        self.tipo = tipo          # "mover"
| "hablar" | "alertar" | "manipular"\n        self.payload = payload\n        self.urgencia
= urgencia  # 0..2 (0 = máxima)\n\nclass CSP:\n    """Capa de Seguridad Física — ÚNICA
vía a actuadores.\n    Valida límites duros y puede vetar o forzar paro."""\n    def
__init__(self):\n            self.emergencia = False\n            self.bloqueadas = 0\n
self.LIMITES = {"velocidad_max": 0.5, "zonas_exclusion": ["escalera", "cocina_fuego"]}\n
```

```
def validar(self, it: Intencion):\n        if self.emergencia:\n            self.bloqueadas
+= 1\n            return False, "PARO DE EMERGENCIA ACTIVO"\n        if it.tipo == "mover"
and it.payload.get("vel", 0) > self.LIMITES["velocidad_max"]:\n            self.bloqueadas
+= 1\n            return False, "VELOCIDAD SOBRE LÍMITE DURO"\n        if it.tipo == "mover"
and it.payload.get("zona") in self.LIMITES["zonas_exclusion"]:\n            self.bloqueadas
+= 1\n                return False, "ZONA DE EXCLUSIÓN"\n            return True, "OK"\n
def paro_emergencia(self):\n            self.emergencia = True\n    def rearmar(self):\n
self.emergencia = False\n\nclass Kernel:\n    """Orquestador: percibir -> evaluar prioridad
-> planificar -> actuar vía CSP."""\n    def __init__(self):\n        self.csp = CSP()\n
self.cola = queue.PriorityQueue()  # (urgencia, seq, Intencion)\n        self.seq = 0\n
self.log = []\n        threading.Thread(target=self._ciclo, daemon=True).start()\n    def
emitir(self, it: Intencion):\n        self.seq += 1\n        self.cola.put((it.urgencia,
self.seq, it))\n    def _ciclo(self):\n        # ~100 Hz: seguridad\n        while True:\n
try:\n                urg, _, it = self.cola.get(timeout=0.01)\n                ok, motivo
= self.csp.validar(it)\n                if ok:\n                    self._ejecutar(it)\n
self.log.append(f"✅ [{it.modulo}] {it.tipo} urg={urg}")\n                    else:\n
self.log.append(f"⛔ [{it.modulo}] {it.tipo} VETADO: {motivo}")\n                self.log =
self.log[-50:]\n            except queue.Empty:\n                pass\n    def _ejecutar(self,
it):\n            # Punto único de actuación (aquí se enchufan motores/altavoz reales)\n
if it.tipo == "hablar":\n                    hablar_fisico(it.payload["texto"])\n\nkernel
= Kernel()\n\n# ============ SENSORES SIMULADOS / ESTADO ROBOT ============\nestado =
{"bateria": 83, "voltaje": 47.2, "geofence": "120m ACTIVO",\n            "temp_cpu": 51.0,
"postura": "NORMAL"}\n\n# ============ MODELOS LOCALES ============\nAPI_CHAT = "http://
localhost:11434/api/chat"\nAPI_GEN  = "http://localhost:11434/api/generate"\nM_TEXTO  =
"phi3:mini"\nM_VISION = "moondream"\nM_EMB      = "all-MiniLM-L6-v2"\n\nprint("Cargando
embedding local...")\nemb = SentenceTransformer(M_EMB)\n\ntry:\n    requests.get("http://
localhost:11434/api/tags", timeout=3)\n    print("✅ Ollama detectado")\nexcept Exception:
\n    print("⚠ Ollama NO responde. Corre: ollama serve")\n\nTTS = False\ntry:\n    import
pyttsx3\n    tts = pyttsx3.init(); tts.setProperty(\'rate\', 175)\n    TTS = True\nexcept
Exception:\n    pass\n\nSTT = False\ntry:\n    from faster_whisper import WhisperModel\n
whisper = WhisperModel("small", device="cpu", compute_type="int8")\n    STT = True\nexcept
Exception:\n    pass\n\ndef hablar_fisico(txt):\n    # Única salida de voz real, invocada
SOLO desde kernel._ejecutar (vía CSP)\n    if not TTS:\n        return\n    def
_h():\n        try:\n            tts.say(txt); tts.runAndWait()\n        except
Exception:\n            pass\n    threading.Thread(target=_h, daemon=True).start()\n\ndef
pedir_hablar(txt):\n        #  pnlio.dialogo.pnl_inversa  emite  Intención;  NO  habla
directo\n    kernel.emitir(Intencion("pnlio.dialogo.pnl_inversa", "hablar", {"texto":
txt},  2))\n\ndef  alertar(txt):\n        #  Prioridad  1:  recordatorios  /  anomalías
leves\n    kernel.emitir(Intencion("kernel.orquestador", "alertar", {"texto": txt},
1))\n\n#  ============  RAG  (pnlio.memoria.contexto)  ============\ndef  chunk_overlap(t,
size=600, overlap=150):\n    out = []\n    for i in range(0, len(t), size - overlap):\n
c = t[i:i + size]\n        if len(c.strip()) > 30:\n            out.append(c)\n        if i
+ size >= len(t):\n            break\n    return out\n\nclass RAG:\n    def __init__(self):
```

```
\n        self.chunks = []; self.metas = []; self.index = None\n    def add(self, name,
text):\n          ch = chunk_overlap(text)\n          if not ch:\n              return
0\n       vecs = np.array(emb.encode(ch), dtype="float32")\n       if self.index is None:
\n           self.index = faiss.IndexFlatL2(vecs.shape[1])\n       self.index.add(vecs)\n
self.chunks.extend(ch); self.metas.extend([name] * len(ch))\n      return len(ch)\n   def
search(self, q, k=4):\n        if not self.index:\n            return []\n        qv =
np.array(emb.encode([q]), dtype="float32")\n        D, I = self.index.search(qv, min(k,
len(self.chunks)))\n        return [(self.metas[i], self.chunks[i]) for i in I[0] if i >=
0]\n\nrag = RAG()\n\ndef extraer(ruta):\n    p = Path(ruta); s = p.suffix.lower()\n    if s
== ".pdf":\n       r = PdfReader(str(p))\n       return "\\n".join([pg.extract_text() or ""
for pg in r.pages])\n    if s == ".docx":\n       d = docx.Document(str(p))\n       partes
= [x.text for x in d.paragraphs]\n       for t in d.tables:\n          for row in t.rows:\n
partes.append(" | ".join(c.text for c in row.cells))\n       return "\\n".join(partes)\n
if s in (".xlsx", ".xls"):\n      h = pd.read_excel(str(p), sheet_name=None)\n      return
"\\n".join([f"[{k}]\\n{v.head(500).to_string()}" for k, v in h.items()])\n      return
p.read_text(encoding="utf-8", errors="replace")\n\n# ============ CHAT ============\ndef
chat_rag(mensaje, history):\n     ctx = rag.search(mensaje)\n     sistema = ("Eres EVA,
sistema robótico asistencial local chilena (PNLIO Kernel v7.11). "\n               "Usa
contexto RAG si existe y cita fuente.")\n    if ctx:\n       sistema += "\\n--- RAG ---\\n"
+ "\\n".join([f"[{n}]\\n{c}" for n, c in ctx])\n    msgs = [{"role": "system", "content":
sistema}]\n    for u, a in history[-4:]:\n       msgs.append({"role": "user", "content":
u})\n        msgs.append({"role": "assistant", "content": a})\n    msgs.append({"role":
"user", "content": mensaje})\n    try:\n       r = requests.post(API_CHAT, json={"model":
M_TEXTO, "messages": msgs, "stream": True},\n                                    stream=True,
timeout=300)\n        full = ""\n        for line in r.iter_lines():\n            if
not line:\n                    continue\n               try:\n                    d =
json.loads(line)\n           p = d.get("message", {}).get("content", "")\n           if
p:\n                           full += p; yield full\n               except Exception:
\n                continue\n    except Exception as e:\n        yield f"Error Ollama:
{e}"\n\ndef subir_doc(file):\n    if not file:\n        return "Sube PDF/Word/Excel"\n
txt = extraer(file.name)\n    n = rag.add(Path(file.name).name, txt)\n    return f"✅
{Path(file.name).name}: {len(txt):,} chars -> {n} chunks. Total: {len(rag.chunks)}"\n\n#
============ pnlio.percepcion.vision ============\ndef ver_camara(imagen, pregunta):\n
if imagen is None:\n           return "Prende la cámara o sube una foto"\n      tmp
= Path(tempfile.gettempdir()) / f"eva_{time.time_ns()}.jpg"\n      try:\n           if
isinstance(imagen, np.ndarray):\n                    Image.fromarray(imagen).save(tmp,
format="JPEG")\n            elif isinstance(imagen, str) and Path(imagen).exists():\n
shutil.copy(imagen, tmp)\n          else:\n               return "Formato de imagen no
soportado"\n    except Exception as e:\n        return f"Error guardando imagen: {e}"\n\n
q = pregunta or ("Describe en español qué ves. IMPORTANTE: reporta si hay personas "\n
"caídas, peligros (fuego, agua, cables), obstáculos para navegación.")\n   try:\n      b64
= base64.b64encode(tmp.read_bytes()).decode()\n      for model in [M_VISION, "llava:phi3",
"llava"]:\n          try:\n             r = requests.post(API_GEN, json={"model": model,
```

```
"prompt": q, "images": [b64],\n                                                    "stream":
False}, timeout=60)\n                if r.status_code == 200:\n                        resp
= r.json().get(\'response\', \'\')\n                        # Detección simple de urgencia -
> Intención prioridad 1\n                    if any(w in resp.lower() for w in ["caída",
"caida", "fuego", "peligro", "tirado"]):\n                        alertar(f"⚠ Posible urgencia
detectada por visión: {resp[:200]}")\n                    return f"👁 [{model}]: {resp}"\n
except Exception:\n                continue\n        return "Falta modelo visión. Corre:
ollama pull moondream"\n    except Exception as e:\n        return f"Error visión: {e}"\n\n#
============ pnlio.percepcion.audio ============\ndef escuchar(audio):\n    if not STT:\n
return "Whisper no instalado: pip install faster-whisper"\n    if not audio:\n        return
""\n    try:\n        segs, _ = whisper.transcribe(audio, language="es")\n        return "
".join([s.text for s in segs]).strip()\n    except Exception as e:\n        return f"Error
STT: {e}"\n\n# ============ COMANDOS DE MOVIMIENTO (siempre vía CSP) ============\ndef
mover(vel, zona):\n    it = Intencion("pnlio.navegacion.nav2", "mover", {"vel": vel,
"zona": zona}, 2)\n    kernel.emitir(it)\n    time.sleep(0.2)  # dar tiempo al ciclo del
kernel\n    return kernel.log[-1] if kernel.log else "sin registro"\n\ndef estado_kernel():
\n    c = kernel.csp\n    return (f"CSP: {\'⛔ EMERGENCIA\' if c.emergencia else \'✅
OK\'} | "\n                f"Vetadas: {c.bloqueadas} | Cola: {kernel.cola.qsize()}\
\n"\n                f"Últimos: " + (" || ".join(kernel.log[-3:]) if kernel.log else
"—"))\n\ndef paro():\n    kernel.csp.paro_emergencia()\n    return estado_kernel()\n\ndef
rearmar():\n    kernel.csp.rearmar()\n    return estado_kernel()\n\n# ============ UI
============\nwith gr.Blocks(title="EVA 2031 v1.7.2", theme=gr.themes.Monochrome()) as
demo:\n    gr.Markdown("# 🤖 EVA 2031 v1.7.2 — SISTEMA ROBÓTICO PNLIO KERNEL v7.11")\n
gr.Markdown("SOBERANO • LOCAL • SEGURO • OFFLINE | Intenciones → CSP → Actuadores")\n
with gr.Row():\n        with gr.Column(scale=2):\n            gr.Markdown("### 💬 CHAT RAG
(pnlio.dialogo)")\n            bot = gr.Chatbot(height=380, type="tuples")\n            txt =
gr.Textbox(label="Tú > escribe o usa el mic", placeholder="Pregunta algo...")\n        with
gr.Row():\n            btn = gr.Button("Enviar", variant="primary")\n            tts_check
= gr.Checkbox(label="🔊 Hablar respuesta (vía CSP)", value=True)\n                clr
= gr.Button("Limpiar")\n        with gr.Column(scale=1):\n            gr.Markdown("###
🛑 SEGURIDAD FÍSICA (CSP)")\n            out_csp = gr.Textbox(value=estado_kernel(),
label="Estado Kernel/CSP", lines=4)\n            with gr.Row():\n                btn_paro =
gr.Button("🛑 PARO DE EMERGENCIA", variant="stop")\n            btn_rearmar = gr.Button("🔄
Rearmar")\n        gr.Markdown("### 🧭 NAVEGACIÓN (vía CSP)")\n        with gr.Row():\n
vel = gr.Slider(0, 1, value=0.3, step=0.05, label="Velocidad (m/s)")\n            zona =
gr.Dropdown(["sala", "dormitorio", "escalera", "cocina_fuego"], value="sala", label="Zona
destino")\n        btn_mover = gr.Button("▶ MOVER (emite Intención)")\n        out_mover
= gr.Textbox(label="Resultado Intención", lines=2)\n    with gr.Row():\n        with
gr.Column(scale=1):\n            gr.Markdown("### 👁 VE - CÁMARA EN VIVO")\n            cam =
gr.Image(sources=["webcam", "upload"], label="Webcam / Foto", height=260)\n        q_vis
= gr.Textbox(label="Qué quieres que vea?", placeholder="¿Hay personas? ¿Peligros?")\n
btn_vis = gr.Button("👁 ANALIZAR LO QUE VEO"); out_vis = gr.Textbox(label="EVA ve:",
```

```
lines=4)\n            with gr.Column(scale=1):\n                gr.Markdown("### 👂
ESCUCHA")\n            aud = gr.Audio(sources=["microphone", "upload"], type="filepath",
label="Micrófono")\n            btn_aud = gr.Button("👂 Transcribir lo que escuché");
out_aud = gr.Textbox(label="Escuché:")\n        gr.Markdown("### 📂 DOCUMENTOS (RAG)")\n
doc = gr.File(label="Sube PDF/Excel/Word"); out_doc = gr.Textbox(label="RAG Status")\n
with gr.Row():\n            gr.Textbox(value=f"LiFePO4 48V {estado[\'bateria\']}%
{estado[\'voltaje\']}V ESTABLE", label="Batería")\n        gr.Textbox(value=f"Geofence
{estado[\'geofence\']}", label="Geofence")\n        gr.Textbox(value="FAISS Index Local",
label="FAISS")\n        gr.Textbox(value=f"Kernel v7.11 | {VERSION}", label="Kernel")\n\n
def enviar(m, h, chk):\n        if not m:\n            yield h, ""\n            return\n
h = h or []\n        full = ""\n        for c in chat_rag(m, h):\n            full = c\n
yield h + [(m, full)], ""\n      if chk:\n          pedir_hablar(full)  # Intención, no acto
directo\n\n    btn.click(enviar, [txt, bot, tts_check], [bot, txt])\n    txt.submit(enviar,
[txt, bot, tts_check], [bot, txt])\n        clr.click(lambda: ([], ""), None, [bot,
txt])\n    btn_vis.click(ver_camara, [cam, q_vis], [out_vis])\n    btn_aud.click(escuchar,
[aud], [out_aud]).then(lambda x: x, [out_aud], [txt])\n    doc.upload(subir_doc, [doc],
[out_doc])\n      btn_paro.click(paro, None, [out_csp])\n      btn_rearmar.click(rearmar,
None, [out_csp])\n    btn_mover.click(mover, [vel, zona], [out_mover])\n\nif __name__ ==
"__main__":\n    demo.launch(server_name="127.0.0.1", server_port=7860)\n'
```

```python
def paso(msg: str) -> None:
    print(f"\n[PNLIO 2031] {msg}")


def ok(msg: str) -> None:
    print(f"[OK] {msg}")


def fallar(msg: str) -> None:
    print(f"[ERROR] {msg}")
    sys.exit(1)


def correr(cmd, shell: bool = False, sudo: bool = False) -> bool:
    """Ejecuta un comando; con sudo=True pide la contraseña una sola vez."""
    if sudo:
        cmd = f"sudo {cmd}" if isinstance(cmd, str) else ["sudo"] + cmd
    r = subprocess.run(cmd, shell=shell or isinstance(cmd, str))
    return r.returncode == 0


def ollama_vivo() -> bool:
```

```
    try:
        with urllib.request.urlopen(PUERTO_OLLAMA, timeout=2) as r:
            return b"Ollama is running" in r.read()
    except Exception:
        return False


def crear_lanzador(nombre: str, destino: Path, extra_env: str = "") -> Path:
    """Crea lanzador bash en CARPETA/nombre y symlink en ~/.local/bin/nombre."""
    lanzador = CARPETA / nombre
    lanzador.write_text("#!/usr/bin/env bash\n"
                        + extra_env
                        + f'exec "{ENTORNO}/bin/python" "{destino}" "$@"\n',
                        encoding="utf-8")
    lanzador.chmod(0o755)
    bin_local = Path.home() / ".local" / "bin"
    bin_local.mkdir(parents=True, exist_ok=True)
    enlace = bin_local / nombre
    if enlace.exists() or enlace.is_symlink():
        enlace.unlink()
    enlace.symlink_to(lanzador)
    return enlace


def crear_escritorio(nombre_archivo: str, titulo: str, comentario: str, exec_cmd: str) -
> None:
    apps = Path.home() / ".local" / "share" / "applications"
    apps.mkdir(parents=True, exist_ok=True)
    (apps / nombre_archivo).write_text(
        "[Desktop Entry]\n"
        f"Name={titulo}\n"
        f"Comment={comentario}\n"
        f"Exec={exec_cmd}\n"
        "Icon=utilities-terminal\n"
        "Terminal=true\n"
        "Type=Application\n"
        "Categories=Utility;\n",
        encoding="utf-8",
    )


def main() -> None:
    print(BANNER)
```

```
    if not Path("/etc/debian_version").exists():
        fallar("este despliegue es para Ubuntu/Debian")
    if shutil.which("sudo") is None:
        fallar("falta sudo; ejecuta desde una cuenta con permisos")

    # 0. Chequeos de espacio y RAM ------------------------------------------
    libre_gb = shutil.disk_usage(Path.home()).free / (1024 ** 3)
    if libre_gb < ESPACIO_MINIMO_GB:
            fallar(f"espacio insuficiente: {libre_gb:.1f} GB libres, se necesitan
{ESPACIO_MINIMO_GB} GB")
    ok(f"Espacio en disco suficiente: {libre_gb:.1f} GB libres")
    ram_gb = 0.0
    try:
        with open("/proc/meminfo") as f:
            for linea in f:
                if linea.startswith("MemTotal"):
                    ram_gb = int(linea.split()[1]) / (1024 ** 2)
                    break
    except Exception:
        pass
    if ram_gb and ram_gb < RAM_AVISO_GB:
       print(f"[AVISO] RAM: {ram_gb:.1f} GB. ADÁN y el Kernel SOLEDAD funcionarán bien;")
      print("       EVA completa (embeddings + whisper + Ollama a la vez) quedará justa.")
    elif ram_gb:
        ok(f"RAM: {ram_gb:.1f} GB")

    # 1. Dependencias del sistema -------------------------------------------
    paso("Actualizando sistema e instalando dependencias base...")
    if not correr("apt-get update -y", sudo=True):
        fallar("fallo apt-get update (revisa conexión y contraseña sudo)")
    if not correr("apt-get install -y curl zstd python3 python3-venv python3-pip espeak-
ng libespeak1", sudo=True):
        fallar("fallo instalando paquetes base")

    # 2. Ollama -------------------------------------------------------------
    CARPETA.mkdir(parents=True, exist_ok=True)
    if shutil.which("ollama") is None:
        paso("Instalando Ollama (servicio local puerto 11434)...")
        if not correr("curl -fsSL https://ollama.com/install.sh | sh", shell=True):
            fallar("fallo instalando Ollama")
    else:
        ok("Ollama ya estaba instalado")
```

```
correr("systemctl enable --now ollama", sudo=True)
paso("Esperando el servicio Ollama (máx 60 s)...")
for _ in range(30):
    if ollama_vivo():
        break
    time.sleep(2)
if not ollama_vivo():
    paso("systemd no levantó Ollama; arrancándolo en segundo plano...")
    subprocess.Popen(["ollama", "serve"],
                     stdout=open(CARPETA / "ollama.log", "w"),
                     stderr=subprocess.STDOUT,
                     start_new_session=True)
    time.sleep(5)
    if not ollama_vivo():
        fallar(f"Ollama no responde (revisa {CARPETA}/ollama.log)")
ok("Servicio Ollama activo (100% local)")

# 3. Modelos ------------------------------------------------------------------
for modelo in (MODELO_TEXTO, MODELO_VISION):
    paso(f"Descargando modelo {modelo} (paciencia si el disco es lento)...")
    if not correr(["ollama", "pull", modelo]):
        fallar(f"fallo descargando {modelo}")
    ok(f"Modelo {modelo} listo y cacheado localmente")

# 4. Entorno virtual + librerías ----------------------------------------------
paso(f"Creando entorno virtual en {ENTORNO} ...")
if not ENTORNO.exists():
    if not correr(["python3", "-m", "venv", str(ENTORNO)]):
        fallar("fallo creando el venv")
pip = str(ENTORNO / "bin" / "pip")
if not correr([pip, "install", "--quiet", "--upgrade", "pip"]):
    fallar("fallo actualizando pip")
if not correr([pip, "install", "--quiet",
               "requests", "pypdf", "python-docx", "openpyxl", "xlrd", "pandas",
               "pyttsx3", "numpy", "opencv-python-headless"]):
    fallar("fallo instalando librerías base (documentos, voz y visión del kernel)")
ok("PDF · Word · Excel · voz (pyttsx3) · visión (OpenCV headless) instalados")
paso("Instalando librerías de EVA (Gradio, FAISS, embeddings, whisper)...")
if not correr([pip, "install", "--quiet",
         "gradio", "faiss-cpu", "sentence-transformers", "Pillow", "faster-whisper"]):
    fallar("fallo instalando librerías de EVA (RAG + dashboard)")
ok("Gradio · FAISS · sentence-transformers · faster-whisper instalados")
```

```
    # 5. Chat ADÁN, Kernel SOLEDAD y Dashboard EVA -------------------------------
    paso("Escribiendo el chat ADÁN 2031...")
    (CARPETA / "chat_adan.py").write_text(CHAT_PY, encoding="utf-8")
    ok(f"Chat en {CARPETA / 'chat_adan.py'}")

    paso("Instalando el Kernel SOLEDAD v1.3 (memoria, voz, visión, núcleo robótico)...")
    KERNEL_DIR.mkdir(parents=True, exist_ok=True)
    KERNEL_ARCHIVO.write_text(KERNEL_PY, encoding="utf-8")
                enlace_adan    =    crear_lanzador("adan",    KERNEL_ARCHIVO,    f'export
PNLIO_MODEL="{MODELO_TEXTO}"\n')
    ok(f"Kernel en {KERNEL_ARCHIVO} · comando 'adan' ({enlace_adan})")

    paso("Instalando el dashboard EVA 2031 v1.7.2 (RAG + CSP + visión + voz)...")
    DASH_ARCHIVO.write_text(EVA_PY, encoding="utf-8")
    enlace_eva = crear_lanzador("eva", DASH_ARCHIVO)
    ok(f"Dashboard en {DASH_ARCHIVO} · comando 'eva' ({enlace_eva})")

    # 6. Lanzadores de escritorio ------------------------------------------------
    crear_escritorio("pnlio2031-chat.desktop", "ADÁN 2031 Chat",
                     "Chat local con Ollama (PDF/Word/Excel)",
                     f"{ENTORNO}/bin/python {CARPETA}/chat_adan.py")
    crear_escritorio("pnlio2031-kernel.desktop", "ADÁN 2031 Kernel SOLEDAD",
                     "Memoria, voz, visión y núcleo robótico (simulación)",
                     str(CARPETA / "adan"))
    crear_escritorio("pnlio2031-eva.desktop", "EVA 2031 Dashboard",
                     "RAG, visión, voz y Capa de Seguridad Física (CSP)",
                     str(CARPETA / "eva"))
    ok("Lanzadores creados en tu menú: 'ADÁN 2031 Chat', 'ADÁN 2031 Kernel SOLEDAD', 'EVA
2031 Dashboard'")

    # 7. Autoprueba del núcleo robótico ------------------------------------------
    paso("Ejecutando la autoprueba del núcleo robótico (simulación)...")
    prueba_ok = correr([str(CARPETA / "adan"), "--autoprueba-robot"])
    if prueba_ok:
        ok("Autoprueba del núcleo robótico superada (15/15)")
    else:
        print("\n[ADVERTENCIA] La autoprueba FALLÓ. No conectes hardware real.")

    print("\n" + "=" * 60)
    print("  DESPLIEGUE COMPLETO — FAMILIA PNLIO 2031 OPERATIVA")
    print(f"  Chat con documentos : {ENTORNO}/bin/python {CARPETA}/chat_adan.py")
    print("  Kernel SOLEDAD      : adan    (memoria, voz, visión, robot en simulación)")
    print("  Dashboard EVA       : eva     (luego abre http://127.0.0.1:7860)")
```

```
    print("  Si 'adan'/'eva' no aparecen: export PATH=\"$HOME/.local/bin:$PATH\"")
    print("  Ollama arranca solo con el sistema (systemd).")
    print("=" * 60)

    # 8. ¿Abrir algo de inmediato? ------------------------------------------------
    try:
        resp = input("\n¿Abrir (c)hat, (k)ernel, (e)va o (N)ada? ").strip().lower()
    except EOFError:
        resp = "n"
    py = str(ENTORNO / "bin" / "python")
    if resp == "c":
        os.execv(py, [py, str(CARPETA / "chat_adan.py")])
    elif resp == "k":
        os.execv(str(CARPETA / "adan"), [str(CARPETA / "adan")])
    elif resp == "e":
        os.execv(str(CARPETA / "eva"), [str(CARPETA / "eva")])


if __name__ == "__main__":
    try:
        main()
    except KeyboardInterrupt:
        print("\n[PNLIO 2031] despliegue interrumpido por el usuario.")
        sys.exit(130)
```